

Library Manager Assignment Report

GitHub URL

<https://github.com/umarmuhdhor/Remix-StockApp.git>

Entity Relationship Design

The application manages two entities: `Author` and `Book`.

`Author` represents a writer in the system. It stores the author's name, email, country, and biography. Email is unique to prevent duplicate author records.

`Book` represents a catalog item. It stores title, ISBN, genre, publication year, price, and a required author reference. ISBN is unique to protect catalog integrity.

The relationship is one author to many books:

- `Author` uses `@OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)`.
- `Book` uses `@ManyToOne(fetch = FetchType.LAZY)` and `@JoinColumn(name = "author\_id", nullable = false)`.

The database is created by Hibernate from the JPA annotations. The H2 database is populated by `src/main/resources/data.sql` with 10 authors and 10 books.

Create Operation

Create screens are implemented with JSP and Spring form tags:

- `/authors/new` displays the author creation form.
- `/books/new` displays the book creation form with an author dropdown.

Controller methods:

- `AuthorController#createAuthor`
- `BookController#createBook`

Both methods bind form data with `@ModelAttribute`, validate it with `@Valid`, and persist data through the service layer. Integrity violations such as duplicate email or duplicate ISBN are caught with `DataIntegrityViolationException` and returned to the JSP as field errors.

Read Operation

Read screens:

- `/authors` displays all authors.
- `/books` displays books with author details.

Controller methods:

- `AuthorController#listAuthors`
- `BookController#listBooks`

The book list uses a custom repository query:

```
```java
@Query("""
 select new com.example.librarymanager.dto.BookAuthorView(...)
 from Book b
```

```

 inner join b.author a
 order by a.name asc, b.title asc
 """)
List<BookAuthorView> findBookAuthorViews();

```

This satisfies the required inner join between the two entities and returns a projection containing both book and author fields.

## ## Update Operation

Update screens:

- ``/authors/{id}/edit``
- ``/books/{id}/edit``

Controller methods:

- ``AuthorController#updateAuthor``
- ``BookController#updateBook``

The update methods load an existing entity by id, copy editable form fields, validate selected relationships, and save changes through the service layer.

## ## Repository Layer

Repositories extend ``JpaRepository``:

- ``AuthorRepository` extends JpaRepository<Author, Long>`
- ``BookRepository` extends JpaRepository<Book, Long>`

Additional query methods:

- ``AuthorRepository#findByEmail``
- ``BookRepository#findByIsbn``
- ``BookRepository#findBookAuthorViews``

## ## Service Layer

Service classes hold business logic:

- ``AuthService``
- ``BookService``

``BookService`` resolves the submitted author id into a managed ``Author`` entity before saving a book. This prevents invalid foreign-key relationships and keeps controller logic focused on HTTP concerns.

## ## View Layer

JSP files are stored under ``src/main/webapp/WEB-INF/jsp``.

- ``authors/list.jsp``
- ``authors/form.jsp``
- ``books/list.jsp``
- ``books/form.jsp``
- ``fragments/navigation.jspf``

The styles are in ``src/main/resources/static/css/styles.css``. The pages use JSTL, Expression Language, and Spring form tags for binding and validation messages.

## ## Testing

Repository tests use `@DataJpaTest`:

- Seeded rows are loaded.
- Finder methods work.
- The inner join projection returns expected book-author rows.

Service tests use JUnit 5 and Mockito:

- `AuthService` find and update behavior.
- `BookService` author resolution, update behavior, missing-author handling, and join projection delegation.

Run tests with:

```
```bash
mvn test
```
```

## ## Screenshots

The application has the following pages ready for screenshots after Java 17 and Maven are installed and the app is started with `mvn spring-boot:run`:

- `http://localhost:8080/books` - joined book and author listing.
- `http://localhost:8080/books/new` - book creation form.
- `http://localhost:8080/authors` - author listing.
- `http://localhost:8080/authors/new` - author creation form.
- `http://localhost:8080/books/{id}/edit` - book update form.
- `http://localhost:8080/authors/{id}/edit` - author update form.

Local screenshot capture could not be completed in this workspace because the machine does not currently have Java, Maven, or Gradle installed.

## ## Challenges Faced

The main design challenge was handling the relationship from the book form. The form submits only the selected author id, while JPA needs a managed `Author` entity. This is handled in `BookService#resolveAuthor`, which looks up the author and rejects missing ids.

Another challenge was making integrity errors readable for users. Duplicate emails and ISBNs are caught in the controller and bound back to the corresponding form field.

Local execution was blocked because this machine does not have a Java runtime or Maven installed. The project is still structured as a standard Maven Spring Boot application and is ready to run on a machine with Java 17 and Maven.